Detailed Design Document (Low-Level Design / LLD)

Project: Face Mask Detection System (`faceMask_detection`)
Author: Tech Lead Subagent • **Status:** Approved Engineering Design

> **Purpose:** This Detailed Design Document defines the internal data flow, component sequence, module-level function algorithms, input/output schemas, error-handling mechanisms, and automated test design for the 4 primary code files before implementation begins.

1. Core Code Architecture (The 4 Files)

The codebase is strictly structured across **4 primary code files**:

```
faceMask_detection/
├── requirements.txt           <-- Package dependencies
│
├── src/
│   ├── utils.py               <-- Module 1: Helper tools (validation & box drawing)
│   ├── detector.py            <-- Module 2: Core AI engine (face detection & mask
classification)
│   └── app.py                 <-- Module 3: Streamlit interactive web dashboard
│
└── tests/
    ├── test_images/           <-- Sample verification photos (with/without mask)
    └── test_detector.py       <-- Module 4: Automated pytest test suite
```

2. End-to-End System Sequence & Data Flow

The diagram below illustrates the exact runtime flow across all modules:

```
User (Browser)          src/app.py            src/utils.py             src/detector.py
     |                      |                      |                        |
     |--- 1. Upload file -->|                      |                        |
     |                      |--- 2. Validate bytes>|                        |
     |                      |<-- 3. True / False --|                        |
     |                      |                      |                        |
     |                      |--- 4. Convert to RGB Matrix                   |
     |                      |                      |                        |
     |                      |--- 5. detect_and_classify_frame(image, thresh)->|
     |                      |                      |                        |-- a.
Run Face Detector
     |                      |                      |                        |-- b.
Crop face ROI
     |                      |                      |                        |-- c.
classify_mask_region
     |                      |                      |                        |
predict Mask / No Mask
     |                      |                      |<-- d. draw_colored_box --|
     |                      |                      |       (draws Green/Red)  |
     |                      |                      |--- e. annotated_image -->|
```

```
    |                      |                                                    |
    |                      |<-- 6. Returns {annotated_image, stats, results}-|
    |                      |
    |                      |--- 7. render_metrics(stats)
    |<-- 8. Display image -|
    |    & summary cards   |
```

3. Module-by-Module Detailed Specifications

Module 1: `src/utils.py` (Validation, Color Math & Drawing Engine)

1.1 `validate_image_bytes(file_bytes: bytes) -> bool`

- **Purpose**: Verifies that uploaded raw bytes form a valid, uncorrupt image file.
- **Algorithm**: Checks byte length > 0, verifies format is in `['JPEG', 'JPG', 'PNG']` using PIL, and calls `image.verify()`. Catches all exceptions to prevent crashes.

1.2 `convert_bgr_to_rgb(image: np.ndarray) -> np.ndarray`

- **Purpose**: Converts OpenCV BGR color space to standard RGB color space so faces don't appear blue in web browsers.

1.3 `draw_colored_box(image: np.ndarray, box: tuple, label: str, confidence: float) -> np.ndarray`

- **Purpose**: Applies color-coded bounding rectangles and text banners directly onto the image matrix.
- **Color Rule**: **Green** `(0, 255, 0)` for `"Mask"`; **Red** `(255, 0, 0)` for `"No Mask"`.
- **Banner text**: `f"{label}: {confidence * 100:.1f}%"`.

Module 2: `src/detector.py` (AI Face Detection & Mask Classification)

2.1 `load_face_detector(model_dir: str = None)`

- **Purpose**: Initializes the face detection model weights into memory.

2.2 `classify_mask_region(face_roi: np.ndarray) -> tuple[str, float]`

- **Purpose**: Analyzes the cropped face region to determine if a protective face mask is worn.
- **Algorithm**: Resizes face ROI to `(224, 224)`, analyzes the lower facial half (mouth/nose) vs upper facial half, and computes mask probability score. Returns `("Mask", score)` or `("No Mask", score)`.

2.3 `detect_and_classify_frame(image: np.ndarray, confidence_threshold: float = 0.70) -> dict`

Purpose: The **Master Coordinator Function** that processes a single image frame end-to-end and packages all results into a clean dictionary.

A. What Goes In (Inputs):

- `image: np.ndarray` → The uploaded photo represented as a pixel matrix.
- `confidence_threshold: float = 0.70` → Sensitivity filter (e.g. 70% threshold).

B. What Happens Inside (4-Step Sequential Pipeline):

1. **Step 1 (Find Faces)**: Scans the image matrix using the face detector and locates bounding coordinates for all faces.
2. **Step 2 (Classify Mask)**: Crops each detected face region of interest (ROI) and runs `classify_mask_region()` to determine if a mask is worn.
3. **Step 3 (Paint Annotations)**: Calls `draw_colored_box()` from `src/utils.py` to paint a **Green box** if masked, or **Red box** if unmasked.
4. **Step 4 (Tally Summary Counters)**: Counts total individuals, compliant masked count, and non-compliant unmasked count.

C. What Comes Out (Output Dictionary Schema):

```
{
    # 1. The image array with the Green and Red boxes painted on it
    "annotated_image": np.ndarray,

    # 2. Total number of faces found in the photo (e.g., 2)
    "total_faces": 2,

    # 3. Total number of people wearing a mask (e.g., 1)
    "masked_count": 1,

    # 4. Total number of people NOT wearing a mask (e.g., 1)
    "unmasked_count": 1,

    # 5. Detailed coordinate list for each detected person
    "results": [
        {
            "box": (100, 150, 200, 250),   # (startX, startY, endX, endY)
coordinates
            "label": "Mask",                # Classification verdict
            "confidence": 0.982             # 98.2% confident
        },
        {
            "box": (350, 160, 450, 260),   # Coordinates of second person
            "label": "No Mask",             # Classification verdict
            "confidence": 0.951             # 95.1% confident
        }
    ]
}
```

D. How the Web Dashboard (`src/app.py`) Consumes This Output:

- Takes `"annotated_image"` → displays the processed photo with boxes on the web page.
- Takes `"total_faces"` → displays on Metric Card 1: **"Total Faces: 2"**.
- Takes `"masked_count"` → displays on Metric Card 2: **"Compliant (Mask): 1"** (in Green).
- Takes `"unmasked_count"` → displays on Metric Card 3: **"Needs Attention: 1"** (in Red).

Module 3: `src/app.py` (Streamlit Interactive Web Dashboard)

3.1 Layout & Sidebar Controls

- **Sidebar Mode Selection**: Radio switch between `"Photo Upload Mode"` and `"Live Camera Mode"`.
- **Confidence Slider**: `st.sidebar.slider("Detection Confidence Threshold", 0.50, 0.95, 0.70, 0.05)`.
- **Metrics Cards**: Renders Total Faces, Masked (Green), and Unmasked (Red).
- **Download Button**: Allows users to save the marked photo to disk.
- **Launch Command**: `streamlit run src/app.py`.

Module 4: `tests/test_detector.py` (Automated QA Test Suite)

4.1 Purpose of the Test Suite

The test suite guarantees that all utility functions, AI detection pipelines, and edge cases function correctly before client release. It runs via: `pytest tests/ -v`.

4.2 Test Data Fixtures (`tests/test_images/`)

- `tests/test_images/person_with_mask.jpg`: Verified portrait wearing a face mask.
- `tests/test_images/person_no_mask.jpg`: Verified portrait with a bare face.

4.3 Test Case Specifications

Test Function	What It Tests	Assertion Rule
`test_validate_image_bytes_valid()`	Upload validation with real image bytes.	`assert validate_image_bytes(valid_bytes) is True`
`test_validate_image_bytes_corrupt()`	Upload validation with corrupt data.	`assert validate_image_bytes(b"corrupt") is False`
`test_draw_colored_box_mask()`	Green box color verification.	Asserts bounding box border is RGB `(0, 255, 0)`.
`test_draw_colored_box_no_mask()`	Red box color verification.	Asserts bounding box border is RGB `(255, 0, 0)`.
`test_detection_on_masked_image()`	Accuracy on `person_with_mask.jpg`.	Asserts `total_faces >= 1` and label is `"Mask"`.

Test Function	What It Tests	Assertion Rule
`test_detection_on_unmasked_image()`	Accuracy on `person_no_mask.jpg`.	Asserts `total_faces >= 1` and label is `"No Mask"`.
`test_zero_faces_image()`	Edge case: blank background photo.	Asserts `total_faces == 0` without crashing.

4.4 Visual Verification Artifact Generation

- `output/result_masked.jpg`: Shows the face enclosed in a **Green box** labeled `Mask: 98%`.
- `output/result_unmasked.jpg`: Shows the face enclosed in a **Red box** labeled `No Mask: 95%`.

4. Error Handling & Edge-Case Matrix

Edge Case Scenario	Potential Failure Point	System Defense Mechanism
Empty or 0-byte file uploaded	Crash during image decode	`validate_image_bytes` checks file length and returns clean error message.
Corrupted / Truncated image	Unhandled exception	Caught in `try...except` block; displays user-friendly warning without breaking session.
Photo with 0 human faces	Loop index out-of-range	Function returns `total_faces = 0`, original image displayed intact with banner: *"No faces detected."*
Photo with crowded group (10+ faces)	Performance lag	Vectorized NumPy array slicing; loops through detected ROIs efficiently.
Face near image boundary	Coordinate negative out-of-bounds	Coordinates are clamped: `startX = max(0, startX)`, `startY = max(0, startY)`.

5. Traceability Matrix: PRD Requirements to Code

PRD Feature	Requirement Description	Implementing Function / File
Feature 1 & 2	Locate faces and classify Mask vs No Mask	`src/detector.py` → `detect_and_classify_frame()`
Feature 3 & 4	Draw Green / Red boxes with percentage label	`src/utils.py` → `draw_colored_box()`
Feature 5	Sidebar mode switch (Upload vs Camera)	`src/app.py` → `st.sidebar.radio()`
Feature 6	Drag & drop photo uploader	`src/app.py` → `st.file_uploader()`
Feature 7	Confidence sensitivity slider	`src/app.py` → `st.sidebar.slider()`
Feature 8	Summary metrics cards (Total, Masked, Unmasked)	`src/app.py` → `render_metrics()`
Feature 9	Download annotated result button	`src/app.py` → `st.download_button()`
Quality Gate	Automated Verification on real images	`tests/test_detector.py` → `pytest`